CARNET DE CONSTRUCTION

CatOpt

Apprendre le C++
avec la théorie des catégories

Un guide pas à pas pour construire ton premier
langage de morphismes et son optimiseur.

Pourquoi cette étape existe dans CatOpt

Tu vas représenter des programmes comme des morphismes, les composer, les exécuter, puis les transformer à l'aide d'équations explicites. Chaque nouvelle notion de C++ répondra à un besoin de cette construction.

$$U \xrightarrow{A_1} U \xrightarrow{S} U \qquad \rightsquigarrow \qquad \mathrm{List}(U) \xrightarrow{F(S \circ A_1)} \mathrm{List}(U)$$

Ajouter un, élever au carré, puis transformer une liste
avec un seul parcours au lieu de deux.

Point de départ : tu as déjà pratiqué les classes et rencontré les smart pointers ; les bases restent à consolider et les templates à découvrir.

Format : tâches courtes, signatures proposées, exemples attendus, questions de compréhension et critères de réussite. Les fonctions centrales restent à écrire par toi.

Volume 1 · Du premier morphisme à la fusion des `map`

Version du 6 novembre 2026

Table des matières

Le fil conducteur : que construit-on exactement ? **1**

Étape 0 — Préparer un atelier minimal **2**

Étape 1 — Donner un sens précis aux calculs **3**

Étape 2 — Représenter les générateurs comme des données **5**

Étape 3 — Construire un morphisme comme un chemin **7**

Étape 4 — Composer et vérifier les lois de catégorie **9**

Étape 5 — Écrire les premières réécritures justifiées **11**

Étape 6 — Passer des scalaires aux listes avec un foncteur **13**

Étape 7 — Fusionner les parcours par fonctorialité **15**

Étape 8 — Consolider le C++ sans perdre le modèle **18**

La démonstration qui termine ce premier volume **20**

La suite : choisir une nouvelle difficulté à la fois **21**

Repères de travail et ressources **22**

Le fil conducteur : que construit-on exactement ?

L'objectif principal est de progresser en C++ en construisant quelque chose qui te plaît. CatOpt sera d'abord un **petit langage de calculs purs** écrit en C++. Il manipulera des descriptions de programmes que tu peux inspecter. Ce volume aboutit à un interpréteur et à deux petites optimisations ; la génération de code C++ viendra dans un volume ultérieur.

Le chemin complet sera :

$$\boxed{\text{Décrire}} \longrightarrow \boxed{\text{Composer}} \longrightarrow \boxed{\text{Interpréter}} \longrightarrow \boxed{\text{Réécrire}}.$$

Le résultat concret que tu vises

Tu construiras le morphisme $f(x) = x + 1$, le morphisme $g(x) = x^2$, puis leur composition $g \circ f$. Tu pourras afficher sa description et vérifier que l'entrée 3 produit 16.

Ensuite, tu compareras ces deux programmes de listes :

$$\underbrace{F(f) \text{ puis } F(g)}_{\text{deux parcours}} \qquad \text{et} \qquad \underbrace{F(g \circ f)}_{\text{un parcours}}.$$

Ils donneront les mêmes valeurs ; le second évitera un tableau intermédiaire dans l'interpréteur que tu auras construit.

Pourquoi commencer par des suites d'opérations ?

Un morphisme sera d'abord une suite comme `[Add(1), Square]`. Tu apprendras ainsi les valeurs, les références, les conteneurs et la composition sans devoir gérer immédiatement un arbre récursif. Cette représentation a une raison mathématique : un morphisme librement construit à partir de générateurs peut être représenté par un chemin.

Nous commencerons avec **un seul objet** U, donc avec des morphismes $U \to U$, appelés *endomorphismes*. C'est déjà une catégorie. Le fait que toutes les compositions soient typées correctement découlera de cette restriction. Le vérificateur de types général arrivera quand nous introduirons d'autres domaines et codomaines.

La notation $[\![p]\!](x)$ signifiera « le résultat de l'interprétation du programme p sur la valeur x ». Elle distingue la description p de la fonction qu'elle représente.

Comment utiliser ce guide

Pour chaque étape : lis l'objectif, écris sur papier l'exemple mathématique, implémente une seule tâche, compile, puis exécute les vérifications. Si tu bloques, isole une question précise : « Que contient ce vecteur ? », « Qui possède cette valeur ? », « Dans quel ordre les fonctions s'appliquent-elles ? »

Point à comprendre

Les blocs de signatures indiquent les fonctions à écrire : ce sont des *déclarations*, pas des implémentations complètes. Une déclaration qui finit par un point-virgule indique au compilateur qu'une fonction existe. Sa définition, avec un corps entre accolades, sera ton travail.

Garde une courte note après chaque étape : ce que tu as construit, la loi mathématique concernée, une notion C++ comprise et une question encore ouverte. Tu peux répartir une étape sur plusieurs séances. Le critère de progression est ta compréhension, pas la vitesse.

Étape 0 — Préparer un atelier minimal

Pourquoi cette étape existe dans CatOpt

Pouvoir compiler et exécuter une petite modification immédiatement. Ce cycle court te permettra de confronter une définition mathématique à son implémentation sans perdre le fil du projet.

C++ à travailler : point d'entrée `main`, en-têtes, compilation, avertissements et assertions.

À implémenter, dans cet ordre

1. Dans `main.cpp`, écris un programme qui affiche `CatOpt - premier morphisme`. Il doit inclure `<iostream>`.

2. Crée un dossier `build`, puis compile depuis la racine du projet. Avec Clang, les commandes sont :

```
mkdir -p build
clang++ -std=c++20 -Wall -Wextra -Wpedantic -g -O0 \
    main.cpp -o build/catopt
./build/catopt
```

   Avec GCC, remplace `clang++` par `g++`. Le dossier `build` contient les exécutables ; ton code source reste à la racine pour l'instant.

3. Ajoute `<cassert>` et une assertion vraie dans `main`. Change-la volontairement en assertion fausse, observe l'arrêt du programme, puis rétablis-la. Tu sais maintenant reconnaître un test qui échoue.

4. Crée une fonction `void run_tests();`, appelle-la depuis `main`, et place tes assertions dans sa définition. Elle accueillera les premiers tests du langage.

Le lien avec la théorie des catégories

Nous voulons vérifier des propriétés des morphismes. L'atelier de tests sera le lieu où tu écriras des exemples de ces propriétés. Une assertion n'est pas une preuve universelle : elle vérifie un cas concret et révèle des erreurs dans l'implémentation.

Questions à te poser

Que fait `##include` ? Quelle différence fais-tu entre une erreur de compilation, une erreur d'édition de liens et un échec à l'exécution ? Peux-tu retrouver le premier message utile quand une compilation échoue ?

Tu peux passer à la suite lorsque…

☐ Tu sais recompiler après une modification.
☐ Le programme s'exécute et une assertion fausse l'arrête.
☐ Tu as compris chaque avertissement restant, et corrigé ceux qui signalent une erreur.
Pour cette phase, conserve les assertions actives : ne définis pas `NDEBUG`, qui les désactive.

Étape 1 — Donner un sens précis aux calculs

Pourquoi cette étape existe dans CatOpt

Définir les valeurs que CatOpt manipule et les premières fonctions qu'il saura représenter. Une optimisation ne peut être justifiée que si l'on sait précisément ce que signifie le programme initial.

C++ à travailler : alias de type, paramètres, retour par valeur, arithmétique non signée et limites d'un type.

Le lien avec la théorie des catégories

Notre premier objet sera $U = \mathbb{Z}/2^{64}\mathbb{Z}$, représenté par les entiers de 0 à $2^{64} - 1$. Les calculs se font modulo 2^{64}. Nous définissons trois familles de fonctions :

$$\mathrm{id}_U(x) = x, \qquad A_c(x) = x + c \pmod{2^{64}}, \qquad S(x) = x^2 \pmod{2^{64}}.$$

Toutes ont le type mathématique $U \to U$.

À implémenter, dans cet ordre

1. Inclus `<cstdint>` et déclare :

   ```
   using Scalar = std::uint64_t;
   ```

   Cet alias donne un nom de projet au type. Il ne crée pas un nouveau type distinct. Ce guide suppose une plateforme qui fournit `std::uint64_t`, comme les plateformes usuelles de Clang et GCC.

2. Écris les trois fonctions suivantes :

   ```
   Scalar identity_value(Scalar x);
   Scalar add_value(Scalar x, Scalar constant);
   Scalar square_value(Scalar x);
   ```

3. Dans les tests, compose manuellement l'addition de 1 et le carré. L'entrée 3 doit donner 16. Inverse l'ordre : tu dois obtenir 10.

4. Inclus `<limits>`. Obtiens la valeur maximale avec `std::numeric_limits<Scalar>::max()` et vérifie que lui ajouter `Scalar{1}` produit zéro.

Exemples attendus

Calcul	Entrée	Résultat
id_U	7	7
A_1	3	4
S	4	16
$S \circ A_1$	3	16
$A_1 \circ S$	3	10
A_1	$2^{64} - 1$	0

Point à comprendre

Le choix du type fait partie de la sémantique. Ici, addition et multiplication sur nos valeurs non signées de 64 bits suivent l'arithmétique modulaire. Nous ne prétendons pas calculer sur tous les entiers mathématiques. Employer un `int` signé et laisser le résultat dépasser son domaine introduirait un comportement indéfini en C++.

Les fonctions du langage sont pures : pas d'affichage, de mutation d'une variable globale ni d'aléatoire dans leur calcul. Les diagnostics de CatOpt seront réalisés autour de l'évaluation. Nous raisonnons sur les résultats mathématiques ; les échecs matériels ou d'allocation sont en dehors du modèle de ce premier volume.

Questions à te poser

Pourquoi passer un petit entier par valeur ? Quelle différence y a-t-il entre le paramètre `x`, une variable locale et la valeur retournée ? Pourquoi l'ordre de composition change-t-il le résultat ?

Tu peux passer à la suite lorsque…

☐ Tous les exemples du tableau sont vérifiés.
☐ Tu peux expliquer ce qu'est l'objet U.
☐ Tu peux expliquer pourquoi $A_1 \circ S \neq S \circ A_1$.

Lien avec la suite : pour l'instant, tu exécutes directement des fonctions C++. L'optimiseur aura besoin d'en manipuler la *description*.

Étape 2 — Représenter les générateurs comme des données

Pourquoi cette étape existe dans CatOpt

Permettre à CatOpt d'inspecter une opération avant de l'exécuter. Pour reconnaître « ajouter zéro », le moteur doit voir le nom de l'opération et sa constante, pas seulement disposer d'une fonction exécutable.

C++ à travailler : `enum class`, `struct`, initialisation, `switch`, `const&`, `std::string`.

Le lien avec la théorie des catégories

Les opérations A_c et S sont nos *générateurs*. Un générateur est un bloc élémentaire à partir duquel nous construirons des morphismes par composition. Sa description est syntaxique ; la fonction $U \to U$ qu'il désigne est son interprétation.

À implémenter, dans cet ordre

1. Déclare un `enum class PrimitiveKind` contenant deux valeurs : `AddConstant` et `Square`.

2. Déclare une `struct Primitive` avec un champ `kind` de ce type et un champ `constant` de type `Scalar`.

3. Écris deux fonctions de construction :

   ```
   Primitive make_add(Scalar constant);
   Primitive make_square();
   ```

   Pour `Square`, fixe `constant` à zéro. Ce champ n'y sert pas, mais une convention stable facilitera les comparaisons de descriptions.

4. Écris l'interpréteur d'une primitive :

   ```
   Scalar evaluate(const Primitive& primitive, Scalar x);
   ```

   Utilise un `switch` sur `kind` et les fonctions de l'étape 1. N'ajoute pas de retour arbitraire comme zéro pour masquer un cas invalide ; tu peux terminer par une exception `std::logic_error` de `<stdexcept>` si aucun cas reconnu n'a été rencontré.

5. Écris une fonction d'affichage qui retourne une chaîne :

   ```
   std::string to_string(const Primitive& primitive);
   ```

   Elle doit produire `Add(3)` ou `Square`. Inclus `<string>`.

Ce que tu dois observer

```
const auto f = make_add(1);
const auto g = make_square();
assert(evaluate(f, 3) == 4);
assert(evaluate(g, 4) == 16);
assert(to_string(f) == "Add(1)");
```

`auto` laisse le compilateur déduire ici le type `Primitive`. Le mot `const` interdit de modifier la variable après son initialisation.

Pourquoi une référence constante ?

Dans `const Primitive&`, le symbole `&` désigne une référence à l'objet fourni par l'appelant. La fonction peut le lire sans en faire une copie, et `const` lui interdit de le modifier par cette référence. La primitive est petite : un passage par valeur serait aussi défendable. Le choix prépare surtout le même contrat de lecture pour les morphismes, qui contiendront plusieurs opérations.

Point à comprendre

La structure publique permet encore de construire des descriptions qui ne respectent pas nos conventions. Pour ce premier noyau, construis les primitives uniquement avec `make_add` et `make_square`. À l'étape 8, tu rendras cette convention explicite dans l'interface.

Questions à te poser

Quelle différence y a-t-il entre `make_add(1)` et `add_value(3, 1)` ? Une description doit-elle contenir son entrée ? Pourquoi l'affichage ne doit-il pas exécuter la primitive ?

Tu peux passer à la suite lorsque...

- ☐ Chaque primitive peut être construite, affichée et évaluée.
- ☐ `evaluate` ne modifie pas la description reçue.
- ☐ Tu distingues « construire le morphisme » et « calculer sa valeur ».

Étape 3 — Construire un morphisme comme un chemin

Pourquoi cette étape existe dans CatOpt

Représenter un programme entier avec plusieurs générateurs. Cette représentation sera l'entrée de l'interpréteur et, plus tard, de l'optimiseur.

C++ à travailler : `std::vector`, boucle sur une collection, propriété par valeur, copie, surcharges et cas vide.

Le lien avec la théorie des catégories

Un chemin de générateurs donne un morphisme :

$$U \xrightarrow{A_1} U \xrightarrow{S} U.$$

Nous stockons les opérations dans **l'ordre d'exécution**, de gauche à droite : `[Add(1), Square]` représente $S \circ A_1$. Le chemin vide représente id_U, puisqu'il laisse son entrée inchangée.

À implémenter, dans cet ordre

1. Inclus `<vector>` et introduis la représentation :

```
struct Morphism {
    std::vector<Primitive> steps;
};
```

2. Écris deux fonctions de construction :

```
Morphism identity();
Morphism lift(Primitive primitive);
```

   La première crée un chemin vide ; la seconde un chemin contenant exactement la primitive fournie. Ici, `lift` signifie seulement « mettre une primitive dans une description de morphisme » ; nous n'introduisons pas une nouvelle structure catégorique avec ce nom.

3. Écris l'évaluation du chemin :

```
Scalar evaluate(const Morphism& morphism, Scalar x);
```

   Pars d'une valeur courante égale à `x`. Pour chaque primitive, remplace la valeur courante par le résultat de son évaluation. Retourne la valeur finale. Le chemin vide doit fonctionner sans traitement artificiel. Les fonctions `evaluate` pour `Primitive` et pour `Morphism` portent le même nom : ce sont des *surcharges*. Le compilateur choisit la fonction selon le type des arguments. Le même principe s'applique à `to_string`.

4. Écris `std::string to_string(const Morphism& morphism);`. Affiche `Id` pour le chemin vide et, par exemple, `Add(1) >> Square` pour les deux étapes précédentes.

5. Copie un morphisme dans une deuxième variable, modifie les étapes de la copie et vérifie que l'original n'a pas changé.

Exemples attendus

```
const Morphism p{{make_add(1), make_square()}};
assert(evaluate(p, 3) == 16);
assert(evaluate(identity(), 3) == 3);
assert(evaluate(lift(make_square()), 5) == 25);
```

Les deux niveaux d'accolades initialisent la structure puis son vecteur. Si cette syntaxe te gêne, commence avec une variable vide et deux appels à `push_back` ; vérifie ensuite que les deux écritures décrivent le même chemin.

Pourquoi aucun smart pointer pour le moment ?

Le `Morphism` possède son vecteur, qui possède ses primitives. Quand le morphisme est détruit, les ressources de son vecteur sont libérées automatiquement. C'est déjà une application du principe RAII : la durée de vie des ressources est attachée à celle des objets.

Un `std::shared_ptr` introduirait ici une question de partage que notre modèle n'exige pas. Les pointeurs intelligents deviendront pertinents si tu choisis ensuite de représenter des expressions récursives ou partagées.

Questions à te poser

Quel objet possède les primitives ? Que copie-t-on en copiant un `Morphism` ? Pourquoi la variable courante de `evaluate` peut-elle changer alors que le morphisme reçu est constant ?

Tu peux passer à la suite lorsque. . .

- ☐ L'identité est un vecteur vide et conserve toute entrée testée.
- ☐ Un chemin est évalué dans son ordre de stockage.
- ☐ Modifier une copie ne modifie pas l'original.
- ☐ Tu peux relier le vecteur au chemin de morphismes dessiné plus haut.

Étape 4 — Composer et vérifier les lois de catégorie

Pourquoi cette étape existe dans CatOpt

Disposer de l'opération centrale du langage : construire un programme à partir de deux programmes. La composition servira ensuite à fusionner les calculs effectués sur les éléments d'une liste.

C++ à travailler : lecture par référence constante, création d'un résultat par valeur, concaténation et égalité de descriptions.

Le lien avec la théorie des catégories

La convention est fixe : `compose(g, f)` représente $g \circ f$, donc **on exécute d'abord f, puis g**. Les lois attendues sont :

$$\mathrm{id}_U \circ f = f, \qquad f \circ \mathrm{id}_U = f, \qquad h \circ (g \circ f) = (h \circ g) \circ f.$$

Elles portent sur la composition. L'associativité de la composition ne signifie pas que toutes les opérations arithmétiques sont associatives.

À implémenter, dans cet ordre

1. Écris :

   ```
   Morphism compose(const Morphism& g, const Morphism& f);
   ```

   Construis un résultat contenant les étapes de `f`, puis celles de `g`. Commence par deux boucles avec `push_back`. Tu pourras ensuite étudier `insert` et `reserve`, sans changer le contrat.

2. Vérifie que la composition ne modifie aucun argument. Retourne le résultat par valeur. Ne retourne jamais une référence vers une variable locale, dont la durée de vie se termine à la sortie de la fonction.

3. Écris une comparaison structurelle explicite :

   ```
   bool same_syntax(const Morphism& a, const Morphism& b);
   ```

   Compare la longueur, les sortes d'opérations et, pour une addition, la constante. Cette fonction compare des descriptions, pas toutes les valeurs qu'elles pourraient produire.

4. Vérifie les deux lois d'identité et l'associativité avec cette comparaison, sur plusieurs chemins, dont le chemin vide.

5. Vérifie aussi l'accord entre composition et interprétation :

   ```
   const auto f = lift(make_add(1));
   const auto g = lift(make_square());
   const Scalar x{3};
   assert(evaluate(compose(g, f), x)
          == evaluate(g, evaluate(f, x)));
   ```

   Utilise notamment `x` égal à 0, 3, 7 et à la valeur maximale.

Une erreur d'ordre que tes tests doivent détecter

Pose $f = A_1$ et $g = S$. Tu dois obtenir :

$$[\![\text{compose}(g, f)]\!](3) = 16, \qquad [\![\text{compose}(f, g)]\!](3) = 10.$$

Tester seulement deux additions ne suffirait pas à détecter une inversion : elles commutent ici. Choisir un exemple discriminant fait partie de la conception d'un bon test.

Pourquoi les lois sont-elles déjà vraies dans la structure ?

La composition concatène des suites. Ajouter la suite vide ne change rien ; concaténer trois suites donne les mêmes éléments dans le même ordre, quel que soit le parenthésage. Notre représentation a déjà oublié les parenthèses inutiles des compositions.

Les chemins forment la catégorie libre à un objet engendrée par nos primitives. « Libre » signifie ici que l'on n'a pas encore identifié les chemins au moyen de relations supplémentaires entre générateurs.

Point à comprendre

`[Add(1), Add(2)]` et `[Add(3)]` sont deux chemins différents, mais ils désignent la même fonction sur U. L'interprétation $p \mapsto [\![p]\!]$ n'est donc pas injective. L'optimiseur exploitera précisément certaines de ces égalités sémantiques.

Tu peux passer à la suite lorsque…

☐ Tu sais expliquer l'ordre des arguments de `compose`.
☐ Les trois lois sont vérifiées structurellement sur tes exemples.
☐ Tu peux justifier ces lois par la concaténation, au-delà des tests.
☐ Tu distingues égalité de syntaxe et égalité de fonctions.

Étape 5 — Écrire les premières réécritures justifiées

Pourquoi cette étape existe dans CatOpt

Transformer une description en une autre, plus courte, sans changer la fonction calculée. CatOpt devient ici un premier optimiseur.

C++ à travailler : parcours et construction d'un vecteur, cas limites, invariant de boucle et contrat d'une fonction.

Le lien avec la théorie des catégories

Nous ajoutons deux familles d'équations :

$$A_0 = \mathrm{id}_U, \qquad A_b \circ A_a = A_{a+b},$$

où $a + b$ est calculé modulo 2^{64}. Ce sont des lois de nos *générateurs arithmétiques*, pas des lois vraies pour tous les morphismes d'une catégorie quelconque.
Si E désigne ces équations, les chemins peuvent être regroupés modulo les égalités qu'elles engendrent, y compris à l'intérieur d'une composition. On obtient un quotient de la catégorie syntaxique. Pour construire ce premier moteur, tu n'as pas besoin d'implémenter les classes du quotient.

Écris d'abord la justification sur papier

Pour toute valeur $x \in U$,

$$[\![A_b \circ A_a]\!](x) = (x + a) + b = x + (a + b) = [\![A_{a+b}]\!](x).$$

L'associativité de l'addition modulaire justifie cette transformation. Pour A_0, la justification est $x + 0 = x$.

Ces égalités restent valides à l'intérieur d'un chemin : remplacer une sous-composition par une fonction égale préserve la composition avec ce qui vient avant et après. C'est la compatibilité de l'égalité avec la composition.

À implémenter, dans cet ordre

1. Déclare :

   ```
   Morphism simplify(const Morphism& input);
   ```

   La fonction retourne une nouvelle description ; elle ne modifie pas `input`. Première version : reconstruis simplement le chemin en omettant chaque `Add(0)`.

2. Ajoute la fusion d'additions *adjacentes*. Une stratégie possible est d'accumuler une constante en attente pendant que tu lis une série d'additions. Quand tu rencontres `Square`, émets l'addition accumulée si elle est non nulle, remets l'accumulateur à zéro, puis émets `Square`.

3. N'oublie pas d'émettre l'addition encore en attente à la fin du chemin. Si sa constante est zéro, aucune primitive ne doit être ajoutée.

4. Vérifie que `simplify` ne modifie pas son entrée et préserve les valeurs calculées sur les exemples ci-dessous.

5. Vérifie qu'une deuxième simplification produit la même syntaxe que la première : c'est l'idempotence de `simplify`.

Exemples attendus

Dans ce tableau, $M = 2^{64} - 1$ et `[]` désigne le chemin vide.

Avant	Après
`[]`	`[]`
`[Add(0)]`	`[]`
`[Add(1), Add(2)]`	`[Add(3)]`
`[Add(1), Add(2), Add(3)]`	`[Add(6)]`
`[Add(1), Square, Add(2)]`	Inchangé
`[Add(M), Add(1)]`	`[]`
`[Add(0), Square, Add(2), Add(3)]`	`[Square, Add(5)]`

Sur l'entrée 3, le dernier chemin vaut 14 avant comme après simplification. Vérifie aussi plusieurs entrées proches de la borne du type.

L'invariant qui aide à programmer

Après avoir lu une partie du chemin, la sortie déjà construite, suivie de l'addition encore en attente, doit calculer la même fonction que la partie lue. Cet énoncé explique pourquoi tu peux différer l'émission des additions. Il t'aide aussi à repérer deux erreurs classiques : oublier la fin du chemin ou laisser une addition traverser un carré.

Point à comprendre

Tu ne peux pas déplacer librement une addition à travers `Square`. Sur 3, ajouter 1 puis élever au carré donne 16, tandis que l'ordre inverse donne 10. Une transformation doit avoir sa propre justification ; la présence d'une composition ne permet pas de permuter ses termes.

Mesurer ce que cette optimisation fait réellement

Définis le coût syntaxique $C(p)$ comme le nombre de primitives du chemin p. Vérifie sur tes exemples que :

$$C(\text{simplify}(p)) \leq C(p).$$

Cela décrit une réduction d'opérations interprétées. Cela ne démontre encore aucun gain de temps sur un programme compilé.

Cette fonction n'est pas un décideur de toutes les égalités entre fonctions sur U. Elle applique seulement les règles que tu lui as données.

Tu peux passer à la suite lorsque…

☐ Les exemples, notamment le chemin vide et le retour modulaire à zéro, sont traités correctement.
☐ Tu justifies les deux règles pour toute entrée x.
☐ Tes tests comparent avant/après et vérifient l'idempotence.
☐ Tu sais distinguer cette justification générale des tests de ton code.

Premier jalon atteint : tu disposes d'un langage, d'une interprétation et d'une transformation qui préserve cette interprétation.

Étape 6 — Passer des scalaires aux listes avec un foncteur

Pourquoi cette étape existe dans CatOpt

Réutiliser chaque morphisme scalaire pour transformer tous les éléments d'une liste. Cette construction prépare directement la fusion des parcours.

C++ à travailler : vecteur d'éléments, entrée en lecture seule, construction d'une sortie, `reserve` et fonctions à plusieurs surcharges.

Le lien avec la théorie des catégories

Le foncteur des listes finies associe à un ensemble A l'ensemble $\mathrm{List}(A)$, et à une fonction $f : A \to B$ la fonction :

$$F(f)([x_1, \ldots, x_n]) = [f(x_1), \ldots, f(x_n)].$$

Les deux lois de foncteur sont :

$$F(\mathrm{id}_U) = \mathrm{id}_{\mathrm{List}(U)}, \qquad F(g \circ f) = F(g) \circ F(f).$$

À implémenter, dans cet ordre

1. Déclare un alias `using Values = std::vector<Scalar>;` . Il représente les listes de valeurs de notre langage.
2. Écris :

   ```
   Values map_values(const Morphism& f, const Values& input);
   ```

   Crée une sortie vide, lis chaque élément de l'entrée, évalue `f` sur cet élément et ajoute le résultat à la sortie.
3. Vérifie que la liste vide est acceptée et que l'entrée n'est jamais modifiée. L'ordre et le nombre des éléments doivent être conservés.
4. Ajoute `reserve(input.size())` sur le vecteur de sortie pour réserver sa capacité. Cela ne crée aucun élément : ne confonds pas `reserve` avec `resize`.
5. Écris des tests pour les deux lois de foncteur. Tu peux comparer directement les vecteurs de scalaires avec `==`.

Exemples attendus

Avec $f = A_1$, $g = S$ et $xs = [0, 1, 2, 3]$,

$$F(f)(xs) = [1, 2, 3, 4], \qquad F(g)(F(f)(xs)) = [1, 4, 9, 16].$$

Les assertions centrales auront cette forme :

```
const Values xs{0, 1, 2, 3};
assert(map_values(identity(), xs) == xs);
assert(map_values(compose(g, f), xs)
       == map_values(g, map_values(f, xs)));
```

Répète ces vérifications avec une liste vide, un singleton et une liste contenant la valeur maximale du type.

La justification tient élément par élément

Pour un indice i, le côté gauche de la seconde loi produit $[\![g \circ f]\!](x_i) = [\![g]\!]([\![f]\!](x_i))$. Le côté droit produit exactement la même valeur à la même position. Les listes ont la même longueur ; elles sont donc égales. La liste vide satisfait également la loi.

Point à comprendre

La loi $F(\mathrm{id}_U) = \mathrm{id}_{\mathrm{List}(U)}$ exprime une égalité de valeurs produites. Appliquer `map_values` à l'identité peut construire une nouvelle liste égale à l'entrée. La loi n'exige ni la même adresse mémoire ni l'absence d'allocation. Ce décalage entre résultat et coût est justement ce que l'optimiseur pourra exploiter.

Précision sur les catégories employées

On peut voir cette construction comme un foncteur de la catégorie des chemins scalaires vers une catégorie à un objet $\mathrm{List}(U)$, dont les morphismes sont les fonctions de listes ainsi obtenues. L'égalité dans la cible est l'égalité des fonctions, pas celle de leurs descriptions.

C'est aussi l'interprétation scalaire suivie du foncteur des listes List : **Set** $\to$ **Set**. Nous ne prétendons pas avoir un endofoncteur sur une petite catégorie possédant seulement U et $\mathrm{List}(U)$: appliquer de nouveau la construction ferait intervenir $\mathrm{List}(\mathrm{List}(U))$, que notre langage ne représente pas encore.

Tu peux passer à la suite lorsque. . .

- ☐ Les valeurs, l'ordre, la longueur et le cas vide sont corrects.
- ☐ Tu sais expliquer ce que F fait à l'objet et aux morphismes.
- ☐ Tu peux démontrer les lois élément par élément.
- ☐ Tu distingues égalité du résultat et coût de l'exécution.

Étape 7 — Fusionner les parcours par fonctorialité

Pourquoi cette étape existe dans CatOpt

Réaliser la première optimisation directement motivée par une loi de foncteur : remplacer plusieurs parcours d'une liste par un seul parcours qui exécute le morphisme composé sur chaque élément.

C++ à travailler : composition de structures, deux niveaux de parcours, séparation entre transformation d'un programme et exécution.

Le lien avec la théorie des catégories

La loi employée est exactement :

$$\boxed{F(g) \circ F(f) = F(g \circ f).}$$

Les deux côtés désignent la même fonction sur les listes. Ils ont cependant deux descriptions et deux stratégies d'exécution différentes dans CatOpt. Le moteur choisira celle qui matérialise moins de listes intermédiaires.

7.1 — Décrire plusieurs parcours successifs

Introduis une structure distincte du morphisme scalaire :

```
struct MapPipeline {
    std::vector<Morphism> stages;
};
```

Chaque élément du vecteur représente *un parcours complet* de la liste. Le pipeline `[f, g]` signifie : appliquer `map_values(f, ...)`, puis `map_values(g, ...)`. Les étapes sont stockées dans l'ordre d'exécution, comme les primitives d'un morphisme.

Écris les fonctions suivantes :

```
Values evaluate(const MapPipeline& pipeline, const Values& input);
std::string to_string(const MapPipeline& pipeline);
```

Pour l'évaluation, commence avec une copie de l'entrée, puis remplace la liste courante par le résultat de chaque `map_values`. Un pipeline vide retourne donc une liste égale à l'entrée. Affiche `IdList` pour le pipeline vide et, par exemple, `Map(Add(1)) >> Map(Square)` pour deux parcours.

7.2 — Construire la description fusionnée

Déclare :

```
MapPipeline fuse_maps(const MapPipeline& input);
```

Procède par petites tâches :

1. Pour une entrée vide, retourne un pipeline vide.
2. Pars d'un morphisme scalaire accumulé égal à l'identité.
3. Pour chaque étape `stage`, mets à jour l'accumulateur par `compose(stage, accumulated)`. Vérifie cet ordre à la main sur `[f, g]` avant d'écrire la boucle.
4. Pour un pipeline non vide, retourne un pipeline contenant un seul morphisme : celui que tu as accumulé. Dans cette première version, conserve cette étape même si son chemin scalaire est vide.
5. Vérifie que l'entrée reste inchangée et qu'aucune liste de données n'est évaluée pendant la fusion : la fonction manipule des descriptions.

Garde d'abord la simplification scalaire de l'étape 5 séparée de cette transformation. Quand chacune fonctionne, tu pourras simplifier le morphisme fusionné. Tu sauras alors quelle transformation explique chaque changement.

7.3 — Vérifier le résultat et l'ordre

Construis $f = A_1$, $g = S$ et la liste $[0, 1, 2, 3]$. Tu dois obtenir :

```
Avant   : Map(Add(1)) >> Map(Square)
Apres   : Map(Add(1) >> Square)
Entree  : [0, 1, 2, 3]
Resultat: [1, 4, 9, 16]
```

Les résultats avant et après doivent être égaux. Avec l'ordre inverse `[g, f]`, le résultat doit être $[1, 2, 5, 10]$. Vérifie également un pipeline vide, un pipeline d'une seule étape, trois étapes et une liste vide. Teste expressément un pipeline contenant seulement `identity()` : après fusion, il doit garder une étape, même s'il calcule la même fonction qu'un pipeline vide. Leurs descriptions restent distinctes.

7.4 — Comprendre le gain sans brouiller la mesure

Pour une liste de longueur N, un pipeline de $k > 0$ étapes effectue k parcours `map_values`, contre un après fusion. Pour notre interpréteur simple, le bilan des *étapes map*, hors copie initiale commune de l'entrée, est le suivant :

Quantité	Avant	Après
Parcours `map_values`	k	1
Visites d'éléments par ces parcours	kN	N
Listes produites par ces parcours	k	1
Listes intermédiaires issues des `map`	$k-1$	0

Le nombre d'opérations scalaires reste le même si l'on ne simplifie pas le morphisme composé : chaque élément subit toujours f, puis g, etc. Les gains portent ici sur les parcours et les listes matérialisées. Une liste vide ne contient aucun élément à visiter et peut n'exiger aucune allocation. « Liste produite » n'est donc pas un comptage exact d'appels à l'allocateur.

Commence par afficher les nombres d'étapes des descriptions. Si tu veux vérifier les parcours réellement exécutés, ajoute ensuite un compteur dans le pilote de l'interpréteur, autour des appels à `map_values`. Ce compteur est un outil d'observation ; il ne fait pas partie du résultat mathématique des morphismes.

Point à comprendre

Ne conclus pas encore « le programme est deux fois plus rapide ». L'interpréteur a ses propres coûts, notamment la lecture des descriptions. Un chronométrage exige un protocole distinct. La conclusion établie ici est précise : mêmes valeurs, moins de parcours et de listes intermédiaires dans cette exécution.

Questions à te poser

Quelle différence y a-t-il entre `Morphism::steps` et `MapPipeline::stages` ? Pourquoi la loi de foncteur autorise-t-elle la fusion, mais pas l'inversion de f et g ? Pourquoi la pureté est-elle essentielle si l'ordre des appels entre éléments change ?

Tu peux passer à la suite lorsque…

- ☐ La fusion conserve l'ordre des fonctions et les résultats.
- ☐ Un pipeline non vide a exactement une étape après fusion.
- ☐ Le pipeline vide reste vide et tous les cas limites passent.
- ☐ Tu peux citer la loi qui justifie la transformation.
- ☐ Tu peux expliquer le gain sans confondre parcours et opérations scalaires.

Étape 8 — Consolider le C++ sans perdre le modèle

Pourquoi cette étape existe dans CatOpt

Rendre ton petit langage plus lisible et plus difficile à utiliser incorrectement, tout en conservant exactement les comportements déjà testés. Cette étape prépare une extension du projet sans en changer le sens.

C++ à travailler : interfaces, invariants, compilation séparée, encapsulation et responsabilité des types.

À faire, dans cet ordre

1. Sépare les déclarations, les définitions et le programme de démonstration. Une organisation suffisante est :

```
CatOpt/
  catopt.hpp     declarations et types du langage
  catopt.cpp     evaluation, composition et transformations
  tests.hpp      declaration de run_tests
  tests.cpp      definition de run_tests et assertions
  main.cpp       appel des tests et demonstration finale
  build/         fichiers produits par la compilation
```

   Protège chaque en-tête contre les inclusions multiples, par exemple avec `##pragma once`. Inclus `catopt.hpp` dans les fichiers qui utilisent ses types et `tests.hpp` dans `main.cpp` et `tests.cpp`.

2. Compile les trois fichiers sources ensemble :

```
clang++ -std=c++20 -Wall -Wextra -Wpedantic -g -O0 \
    main.cpp catopt.cpp tests.cpp -o build/catopt
```

   Une déclaration dans l'en-tête permet d'utiliser une fonction ; sa définition dans `catopt.cpp` fournit le code associé lors de l'édition de liens.

3. Regroupe les noms du langage dans `namespace catopt`. Actualise les déclarations, les définitions et leurs utilisations.

4. Renforce `Primitive` : empêche désormais la construction de combinaisons arbitraires de champs. Tu peux en faire une classe avec constructeur privé, fabriques publiques statiques et accès en lecture. Adapte les appels et les tests avant de modifier un autre type.

5. Pour `Morphism`, décide quelles opérations doivent avoir accès au stockage. N'ajoute pas automatiquement un setter pour chaque champ. Une interface de lecture et les opérations de construction peuvent suffire.

Le lien avec la théorie des catégories

L'encapsulation protège les représentations sur lesquelles tes lois reposent. Par exemple, une primitive bien construite désigne toujours une opération reconnue, et un morphisme garde un ordre d'exécution explicite. Une classe n'est pas à elle seule une preuve mathématique ; elle aide à faire respecter les hypothèses de ton modèle par le code appelant.

Une lecture mémoire utile

Choisis une fonction et explique la durée de vie de chaque objet qu'elle utilise. Identifie une copie nécessaire et une copie éventuellement évitable. Ne remplace pas chaque retour par `std::move` : le retour par valeur permet déjà, selon la situation, l'élision de copie ou le déplacement. Comprends d'abord le contrat de propriété.

Avec Clang ou GCC sur une plateforme qui les prend en charge, tu peux aussi recompiler avec `-fsanitize=address,undefined -fno-omit-frame-pointer` et exécuter tes exemples. Ces outils recherchent certaines erreurs mémoire et certains comportements indéfinis ; ils ne prouvent pas les lois de CatOpt.

Tu peux passer à la suite lorsque. . .

- ☐ Le programme compile depuis ses différents fichiers.
- ☐ Tous les exemples précédents ont le même résultat.
- ☐ Tu peux expliquer les conventions devenues des invariants protégés.
- ☐ Tu sais qui possède chaque vecteur et pourquoi ses éléments restent valides.

La démonstration qui termine ce premier volume

Prépare un exécutable qui raconte la transformation en quatre temps. Son affichage peut ressembler à ceci :

```
1. Un morphisme scalaire
   f = Add(1)
   g = Square
   g o f = Add(1) >> Square
   (g o f)(3) = 16

2. Une reecriture arithmetique
   Avant : Add(1) >> Add(2) >> Square
   Apres : Add(3) >> Square
   Loi   : A_b o A_a = A_(a+b)
   Entree 3 : 36 dans les deux cas

3. Une reecriture par fonctorialite
   Avant : Map(Add(1)) >> Map(Square)
   Apres : Map(Add(1) >> Square)
   Loi   : F(g) o F(f) = F(g o f)
   Entree : [0, 1, 2, 3]
   Resultat : [1, 4, 9, 16]
   Parcours map : 2 -> 1

4. Verification
   Tous les tests executes sont passes.
```

Cet affichage est une démonstration explicative. Ce n'est pas encore un certificat vérifié indépendamment : les règles et l'interpréteur restent des éléments de confiance de ton programme.

Les cinq questions auxquelles tu dois savoir répondre

1. Quel est l'objet de la catégorie scalaire, et que représentent ses morphismes ?
2. Pourquoi l'identité est-elle une suite vide ?
3. Pourquoi `compose(g, f)` stocke-t-il d'abord `f` ?
4. Quelle différence sépare une loi de catégorie, une loi arithmétique et une loi de foncteur ?
5. Qu'est-ce qui est préservé par l'optimiseur, et quel coût diminue effectivement dans ta première démonstration ?

Pourquoi cette étape existe dans CatOpt

À ce stade, CatOpt possède une syntaxe, une sémantique, une composition, des tests de lois et deux sortes de réécritures. Tu as travaillé les bases du C++ dans un programme cohérent, et tu peux expliquer chaque transformation avec les objets mathématiques qui l'ont motivée.

La suite : choisir une nouvelle difficulté à la fois

Le noyau par chemins est volontairement limité : il décrit des calculs linéaires sur un seul type scalaire, puis des parcours de listes. Tu peux le faire évoluer lorsque tu sais expliquer ses limites concrètes.

Prochain besoin	Notions C++	Question catégorique
Plusieurs objets et types intermédiaires	Types explicites, validation, `std::variant`	Quand $g \circ f$ est-il bien typé ?
Une syntaxe avec `Compose`, `Map` et produits imbriqués	Arbres, visiteurs, récursivité, `std::unique_ptr`	Comment conserver la structure d'un morphisme composé ?
Des sous-expressions partagées	Identifiants, arène, propriété partagée si nécessaire	Comment plusieurs chemins réutilisent-ils une description ?
Une interface typée à la compilation	Templates, déduction, concepts	Peut-on refuser une composition dont les objets intermédiaires diffèrent ?
Deux calculs indépendants	Types produits, représentation de graphes	Que signifie $f \otimes g$, et quand la loi d'interchange est-elle valide ?
Explorer plusieurs réécritures	Hachage, union-find, e-graphs	Comment représenter plusieurs morphismes égaux modulo des équations ?

Une première expérience de templates, au bon moment

Quand le noyau fonctionne, écris à part une fonction de composition de deux objets appelables, paramétrée par leurs types. Compare-la à `compose(const Morphism&, const Morphism&)`. La première construit un calcul exécutable ; la seconde construit une description inspectable. Comprendre cette différence t'évitera de perdre la capacité de réécriture en remplaçant trop tôt toutes les descriptions par des fonctions opaques.

Un premier backend

Une autre suite naturelle consiste à produire un fichier C++ à partir du morphisme fusionné. Tu pourras alors vérifier qu'une seule boucle est générée, compiler le fichier et comparer ses résultats à ceux de l'interpréteur de référence. Le passage du modèle mathématique au code généré deviendra une nouvelle obligation de correction.

Conseil de progression : choisis une seule de ces extensions pour le prochain jalon. Le but reste de comprendre du C++ à travers les constructions catégoriques que tu as envie d'explorer.

Repères de travail et ressources

Petit dictionnaire du projet

Objet
Un domaine de valeurs. Ici, U, puis $\mathrm{List}(U)$.
Morphisme
Une transformation avec un domaine et un codomaine. Nos morphismes scalaires vont tous de U vers U.
Générateur
Une opération élémentaire, ici A_c ou S.
Composition
Enchaînement : $g \circ f$ exécute f, puis g.
Identité
Morphisme qui laisse la valeur inchangée ; chemin vide dans notre syntaxe scalaire.
Interprétation
Passage d'une description p à la fonction $[\![p]\!]$ qu'elle représente.
Foncteur
Application sur les objets et les morphismes qui préserve l'identité et la composition.
Réécriture
Remplacement d'une description par une autre selon une règle justifiée, éventuellement à l'intérieur d'un programme.
Modèle de coût
Critère explicite pour comparer les descriptions : nombre de primitives, de parcours, etc.

La fiche à remplir après chaque séance

1. **Dans CatOpt :** qu'est-ce que mon programme sait faire de plus ?
2. **En catégories :** quel objet, morphisme ou quelle loi ai-je manipulé ?
3. **En C++ :** quelle notion ai-je utilisée, et pourquoi ?
4. **Vérification :** quel exemple pouvait révéler une erreur ?
5. **Prochaine petite tâche :** quelle fonction ou quel cas vais-je traiter ?

Lectures ciblées, facultatives

— Pour vérifier la sémantique des entiers : brouillon de travail du standard C++, types fondamentaux, notamment le paragraphe sur l'arithmétique non signée. Ce document technique évolue ; le comportement modulaire utilisé ici est établi en C++20.

— Pour voir une bibliothèque qui manipule objets, morphismes et diagrammes : documentation de DisCoPy. Elle propose notamment des diagrammes, des compositions et des interprétations par foncteurs. Elle est une source d'exemples, pas un prérequis pour ce guide.

— Après les premières réécritures, pour découvrir la saturation d'égalités : Willsey et al., *egg: Fast and Extensible Equality Saturation*, POPL 2021. Cette lecture prépare une extension ultérieure.

Compiler ce document

Le fichier `CatOpt-guide-demarrage.tex` est autonome et peut être importé dans Overleaf avec le compilateur pdfLaTeX. Avec une installation TeX Live ou MacTeX incluant les paquets utilisés, place-toi dans son dossier :

```
latexmk -pdf -interaction=nonstopmode -halt-on-error \
    CatOpt-guide-demarrage.tex
```

Ta première séance : termine l'étape 0, puis écris les trois fonctions scalaires de l'étape 1. Arrête-toi quand tu sais expliquer pourquoi $S \circ A_1$ envoie 3 sur 16, et pourquoi l'ordre inverse donne 10.